

Building Software with AI Without Creating Chaos

How to move from fast prototype to maintainable AI-assisted product without losing the learning speed.

Core idea

Prototype to learn. Freeze to build. Constrain AI to bounded jobs. Turn every real failure into permanent coverage. The biggest risk is not using AI too much; it is letting fast AI-generated changes outrun clear architecture, ownership, and tests.

The phase change that matters

Phase	Main goal	How AI should help	Exit gate
Prototype	Discover the product and interaction model.	Explore quickly, try variants, generate disposable code, challenge assumptions.	You can explain what the product is and which behaviors matter.
Transition	Convert learning into a build contract.	Audit the prototype, identify invariants, map ownership, define source-of-truth paths.	Golden behaviors, architecture boundaries, and runtime entypoints are explicit.
Real build	Implement reliably without losing the product lessons.	Build bounded slices, write tests, run QA, remove conflicts instead of layering patches.	Automated tests + deployed browser smoke checks pass.

State lesson: The prototype was useful because it made the product concrete. The expensive part came when prototype-era code patterns were allowed to keep evolving after State had become a real backend + database + model product.

Before AI writes production code

- 1. One source of truth:** exact frontend, backend, schema, migration, and deploy paths.
- 2. Behavioral contract:** what must always be true, regardless of model output.
- 3. Golden behaviors:** 8-12 representative journeys with expected outcomes written first.
- 4. Ownership map:** which layer owns routing, validation, persistence, rendering, and authority.
- 5. Provider spike:** test real schema support, latency, cost, and failure modes before deep integration.
- 6. Smoke gate:** the few browser journeys that prove the product is basically usable.

Repository clarity is part of prompt engineering. If two files look equally authoritative, an AI coding assistant can confidently change the wrong one. Clean paths and explicit ownership reduce both human and model mistakes.

Keep the architecture boring

Good complexity

Product rules, authority boundaries, meaningful State transitions, user-facing behavior discovered through use.

Bad complexity

Duplicate runtime paths, patch-on-patch CSS, multiple visibility mechanisms, stale configs, hidden fallbacks, unclear routing.

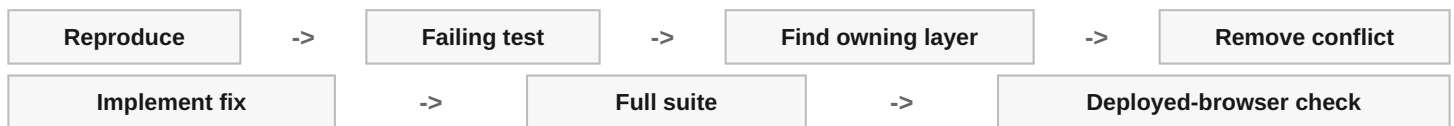
Building Software with AI Without Creating Chaos

A reusable operating model for AI-assisted product development.

Use AI as four different jobs

Job	Objective	Useful instruction
Architect	Find the cleanest owning layer before code changes.	"Identify the source of truth, invariants, dependencies, and simplest design before changing code."
Builder	Implement one bounded slice.	"Implement only this behavior. Preserve the contract. Prefer removing conflicts to adding overrides."
QA	Assume the builder missed something.	"Do not fix yet. Try to break this, inspect adjacent paths, and identify what an automated test should catch."
Product	Judge whether the finished behavior is actually useful.	"Use this like a real user. Separate product confusion from implementation defects and model failures."

The preferred repair loop



Rule: A bug that mattered once should not depend on someone remembering to manually check it forever.

Three State failures worth remembering

Symptom	Real cause	Reusable lesson
Ask gave the wrong factual answer	The question went down the wrong frontend execution path; the model was not the core failure.	Before prompt-engineering a bad answer, verify routing, retrieval, context, and authority status.
The deployed page opened behind a broken modal	Multiple CSS/JS visibility mechanisms and accumulated overrides created conflicting state.	Do not patch a symptom by adding another mechanism. Pick one owner and one source of truth.
AI calls felt painfully slow	Provider inference dominated latency; backend validation was comparatively cheap.	Instrument first. When latency cannot be eliminated, use interaction design and true streaming rather than theatrical progress.

AI-specific build rules

- **Models make semantic judgments; software should own mechanical facts** whenever it can derive them deterministically.
- **The provider's API shape should not define the product contract.** Adapt provider output to the application, not the reverse.
- **Valid structured output is not necessarily correct behavior.** Schema checks and semantic/product checks solve different problems.
- **Do not give the model authority just because it has capability.** Recommend, validate, authorize, then mutate.
- **Use real product usage to discover good rework.** Do not confuse product learning with avoidable codebase entropy.

Useful iteration vs. avoidable debt

Useful rework

"Using this changed our understanding of what the product should do." Keep doing this.

Avoidable rework

"We fixed the same class of implementation problem again because ownership or coverage was unclear." Change the process.

One-sentence summary

Move fast while discovering the product, then make the implementation boring: one source of truth, bounded AI jobs, explicit phase gates, permanent regression coverage, and deployed-browser verification.